

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Департамент программной
инженерии ФКН НИУ ВШЭ
Научный руководитель,
кандидат технических наук,
доцент департамента
программной инженерии
факультета компьютерных наук

_____ / Н.С. Белова /
«__» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Техническое задание

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.02-06 ТЗ 01–1 ЛУ

Исполнители:
студент группы БПИ233

_____ / Б.А. Багавиев /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.02-06 ТЗ 01–1 ЛУ

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Техническое задание

RU.17701729.02-06 ТЗ 01–1

Листов 28

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
1.1. Наименование темы разработки	1
1.2. Наименование программы	1
1.3. Состав комплекса программ	1
1.4. Краткая характеристика области применения	2
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ	3
2.1. Документы, на основании которых ведётся разработка	3
2.2. Наименование темы разработки	3
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ	4
3.1. Функциональное назначение программы	4
3.2. Эксплуатационное назначение программы	5
3.2.1. Архитектура развёртывания	5
3.2.2. Требования к окружению	5
3.2.3. Сценарии использования	6
4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ	7
4.1. Требования к функциональным характеристикам	7
4.1.1. Управление темами (Management API)	7
4.1.2. Управление кластером (Management API)	7
4.1.3. Публикация сообщений (Data API)	8
4.1.4. Чтение сообщений (Data API)	8
4.1.5. Группы потребителей (Data API)	9
4.1.6. Алгоритм диапазонного назначения партиций	9
4.1.7. Аутентификация	9
4.2. Требования к надёжности	10
4.3. Требования к составу и параметрам технических средств	10
4.4. Требования к информационной и программной совместимости	11
4.5. Специальные требования	11
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	12
5.1. Состав программной документации	12
5.2. Специальные требования к документации	12
5.2.1. Требования к оформлению	12
5.2.2. Требования к содержанию пояснительной записки	13
5.2.3. Требования к руководству оператора	13
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	14
6.1. Экономическая эффективность	14
6.1.1. Снижение лицензионных затрат	14
6.1.2. Снижение требований к инфраструктуре	14
6.1.3. Прогнозируемые технические характеристики	14

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ	16
7.1. Сроки разработки и исполнители	17
8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ	18
8.1. Критерии приёмки	18
8.2. Процедура приёмки	19
8.3. Требования к результатам приёмки	20
ManagementService	24
DataService	24
Коды ошибок	25

1. ВВЕДЕНИЕ

1.1. Наименование темы разработки

Наименование темы разработки — «**Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием**».

1.2. Наименование программы

Наименование программы — «**Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием**» (далее по тексту — «Брокер», «Система», «BunnyMQ»).

Наименование программы на английском языке — «**Go message queue with delivery guarantees and horizontal scaling**».

Краткое наименование: **BunnyMQ**.

Обозначение программного документа по ГОСТ 19.103-77: **RU.17701729.02-06 ТЗ 01–1** (числовой код вида документа по ГОСТ 19.101-77: **05**).

Вид программного изделия по ГОСТ 19.101-77, §2.2: **комплекс программ** — совокупность взаимодействующих программных компонентов, объединённых в единую систему: серверный процесс брокера, клиентская библиотека и утилита командной строки.

1.3. Состав комплекса программ

Комплекс BunnyMQ включает три исполняемых компонента и одну библиотеку:

1. `bunmq (cmd/bunmq)` — процесс брокера; запускает gRPC-серверы, узел Raft, координаторы и HTTP-эндпоинт метрик (Go 1.25);
2. `bunmq-cli (cmd/bunmq-cli)` — утилита командной строки для управления кластером и тестирования: команды `produce`, `consume`, `topic`, `cluster` (Go 1.25);
3. `pkg/client` — публичная Go-библиотека для встраивания в клиентский код: типы `Producer`, `Consumer`, `AdminClient` с автоматическим обнаружением лидера и повторными попытками (Go 1.25).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Серверные компоненты, используемые внутри брокера, не являются самостоятельными исполняемыми файлами, а входят в состав пакета `bunmq`.

1.4. Краткая характеристика области применения

BunnyMQ предназначен для применения в инфраструктуре микросервисных приложений, где требуется надёжная асинхронная передача сообщений между компонентами системы с гарантиями доставки и возможностью горизонтального масштабирования.

Типовые сценарии использования:

- передача событий между микросервисами с гарантией «хотя бы однократной» доставки (`acks=all`);
- сбор данных телеметрии, логов или метрик от большого числа производителей с минимальными задержками (`acks=0`);
- организация нескольких независимых потоков данных в рамках одного кластера через механизм тем и партиций;
- параллельная обработка данных несколькими потребителями через группы потребителей с автоматическим перераспределением партиций.

Категории пользователей системы:

- **Разработчики приложений-производителей** — подключаются к кластеру BunnyMQ через клиентскую библиотеку или gRPC напрямую, отправляют сообщения в темы с выбором уровня подтверждения;
- **Разработчики приложений-потребителей** — читают сообщения из партиций начиная с заданного смещения, при необходимости координируют работу через группы потребителей;
- **Администраторы кластера** — управляют топологией кластера через `bunmq-cli` или Management API: создают и удаляют темы, меняют настройки хранения, просматривают состояние кластера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

2.1. Документы, на основании которых ведётся разработка

Разработка ведётся на основании следующих документов:

1. Устав федерального государственного автономного образовательного учреждения высшего образования «Национальный исследовательский университет «Высшая школа экономики»» (с изменениями) № 56 от 1.02.2016.
2. Приказ декана ФКН НИУ ВШЭ И. В. Аржанцева № 2.3-02/1112–04 от 11.12.2025 «Об утверждении тем, руководителей курсовых работ студентов образовательной программы Программная инженерия факультета компьютерных наук».

Организация, утвердившая документ-основание разработки: **Факультет компьютерных наук НИУ ВШЭ.**

Дата утверждения документа-основания: **11.12.2025.**

2.2. Наименование темы разработки

Наименование темы разработки — «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение программы

BunnyMQ реализует следующие функции:

1. **Управление темами** (Topic management) — создание темы с указанием числа партиций, фактора репликации и параметров хранения; удаление темы с асинхронной физической очисткой данных; получение списка тем и описания конкретной темы (метаданные, лидеры партиций, расположение реплик); изменение числа партиций (только увеличение) и параметров хранения (`retention_ms`, `retention_bytes`);
2. **Управление кластером** — получение информации о составе кластера: список узлов, их адреса и статус; список партиций темы с указанием лидера и реплик; состав кластера задаётся статически при запуске;
3. **Публикация сообщений** — приём одного сообщения или пакета сообщений (batch) и запись в указанную партицию. Режим `acks=0`: производитель не ожидает подтверждения записи (реализован через `dragonboat.Propose`). Режим `acks=all`: производитель ждёт подтверждения от кворума реплик партиции (реализован через `dragonboat.SyncPropose`), после чего получает назначенное смещение. Режим `acks=1` намеренно не реализован из-за несовместимости с семантикой Raft;
4. **Чтение сообщений** — получение записей из партиции начиная с заданного смещения и до `max_bytes` байт; при отсутствии новых записей запрос удерживается до `max_wait_ms` миллисекунд (long-poll fetch). Данная функция работает только на узле-лидере партиции;
5. **Запросы смещений** — получение наименьшего (`earliest`) и наибольшего (`latest`) допустимых смещений партиции; поиск наименьшего смещения, чья метка времени не меньше заданной;
6. **Группы потребителей** — регистрация потребителя в группе (`JoinGroup`) с назначением идентификатора члена и текущего поколения группы; периодическое подтверждение активности (`Heartbeat`); выход из группы (`LeaveGroup`); фиксация обработанного смещения (`CommitOffset`); чтение

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

зафиксированных смещений (`FetchCommittedOffsets`). При изменении состава группы инициируется ребалансировка. Стратегия — диапазонное назначение (`range assignment`);

7. **Аутентификация** — клиент передаёт статический токен в gRPC-метаданных (`bunnymq-auth-token`). Сервер проверяет токен по настроенному при запуске списку. Режим PLAINTEXT (пустой список токенов) разрешён и является режимом по умолчанию;
8. **Мониторинг** — экспорт метрик в формате Prometheus на каждом узле кластера.

3.2. Эксплуатационное назначение программы

3.2.1. Архитектура развёртывания

BunnyMQ запускается как кластер из нечётного числа узлов (типично 3 или 5). Каждый узел — один процесс `bunnymq` с доступом к выделенному дисковому разделу для хранения журналов партиций.

Внутри кластера работает протокол Raft, реализованный библиотекой `dragonboat v4` в модели `multi-raft`: один `shard Raft` на каждую партицию плюс один `shard` для метаданных кластера. Клиент при обращении к партиции, лидер которой находится на другом узле, получает ответ `NotLeader` с адресом лидера и повторяет запрос напрямую.

3.2.2. Требования к окружению

- Операционная система: Linux (ядро 5.4+) или macOS 12+, платформа amd64 или arm64;
- Версия Go: 1.25.6 и выше (для сборки из исходного кода);
- Дисковое хранилище: рекомендуется SSD; файловая система — `ext4`, `xfс` или `apfs`;
- Сеть: TCP-связность между всеми узлами кластера на портах Raft и gRPC.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.3. Сценарии использования

Производитель: создаёт подключение к любому узлу кластера через `pkg/client.NewProducer`; библиотека самостоятельно находит лидера нужной партии и отправляет туда сообщение. При `acks=all` производитель получает подтверждённое смещение; при `acks=0` ответ возвращается немедленно без смещения.

Потребитель без группы: вызывает `Fetch` напрямую, указывая тему, партию и смещение; данные читает в темпе своей обработки. Фиксация смещений опциональна.

Потребитель с группой: вызывает `Subscribe` с идентификатором группы; библиотека выполняет `JoinGroup` и получает назначенные партии. В фоне работает горутина `heartbeat`; при сигнале о ребалансировке в ответе `heartbeat` потребитель повторяет `JoinGroup` и получает новое назначение. Обработанные смещения фиксируются через `Commit`.

Администратор: использует `bunmq-cli` или `AdminClient` для управления темами и просмотра состояния кластера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ

4.1. Требования к функциональным характеристикам

4.1.1. Управление темами (Management API)

Брокер должен реализовывать операции управления темами через Management API (gRPC-сервис ManagementService):

- Создание темы (CreateTopic):** входные параметры — name (уникальное имя, регулярное выражение $[a-zA-Z0-9._-]\{1,255\}$), partition_count (≥ 1), replication_factor ($\geq 1, \leq$ числа узлов кластера), опциональные параметры хранения (retention_ms, retention_bytes). При уже существующей теме возвращается код TopicAlreadyExists.
- Удаление темы (DeleteTopic):** физическое удаление данных выполняется асинхронно; при отсутствии темы возвращается TopicNotFound.
- Список тем (ListTopics):** возвращает список всех тем с полями name, partition_count, replication_factor.
- Описание темы (DescribeTopic):** возвращает полные метаданные темы, включая для каждой партиции: идентификатор лидера и список идентификаторов реплик с их статусом.
- Изменение числа партиций (AlterTopicPartitions):** разрешено только увеличение числа партиций; уменьшение не поддерживается.
- Изменение параметров хранения (AlterTopicRetention):** обновляет retention_ms и / или retention_bytes; новые параметры применяются на следующем цикле очистки.

4.1.2. Управление кластером (Management API)

- Описание кластера (DescribeCluster):** возвращает список узлов с полями node_id, address, status.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Список партиций темы** (`ListPartitions`): возвращает для каждой партиции `partition_id`, `leader_node_id`, `replica_node_ids`, диапазон смещений (`earliest_offset`, `latest_offset`).
3. Состав кластера задаётся статически в конфигурационном файле при запуске; добавление и удаление узлов в `runtime` в первой версии не поддерживается.

4.1.3. Публикация сообщений (Data API)

1. **Produce** (`Produce`): входные параметры — `topic`, `partition_id` (если не задан — выбирается по хешу ключа FNV-1a 32 или `round-robin`), `batch_data` (байтовая последовательность в формате Kafka-совместимого пакета), `acks` (0 или `all`). При `acks=all` возвращается назначенное смещение первого сообщения пакета; при `acks=0` возвращается ответ без смещения немедленно.
2. Все сообщения одного пакета записываются в одну партицию с последовательными смещениями атомарно (принцип «всё или ничего»).
3. Формат `batch_data` на диске совпадает с форматом на проводе; клиент кодирует пакет, сервер хранит его без изменений.

4.1.4. Чтение сообщений (Data API)

1. **Fetch** (`Fetch`): входные параметры — `topic`, `partition_id`, `offset`, `max_bytes`, `max_wait_ms`. Возвращает записи с последовательными смещениями начиная с `offset` и до исчерпания `max_bytes`. Если записей нет, запрос удерживается до появления данных или истечения `max_wait_ms`.
2. **GetOffsets** (`GetOffsets`): возвращает `earliest_offset` и `latest_offset` для заданной партиции; поддерживается запрос смещения по метке времени (`GetOffsetByTimestamp`).
3. Запросы `Fetch` и `GetOffsets` обслуживаются только на узле-лидере партиции; при обращении к не-лидеру возвращается `NotLeader` с адресом лидера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.1.5. Группы потребителей (Data API)

1. **JoinGroup**: потребитель указывает `group_id` и список тем. Сервер добавляет потребителя к группе, выполняет диапазонное назначение партиций, возвращает `member_id`, `generation_id` и список назначенных партиций.
2. **Heartbeat**: потребитель периодически (рекомендуется раз в 3 секунды) отправляет `heartbeat` с `group_id`, `member_id`, `generation_id`. Если `generation_id` устарел (произошла ребалансировка), ответ содержит сигнал `REBALANCE_REQUIRED` — потребитель должен повторить `JoinGroup`.
3. **LeaveGroup**: потребитель явно покидает группу; происходит немедленная ребалансировка с новым `generation_id`.
4. **CommitOffset**: потребитель фиксирует смещения по одной или нескольким партициям. Для фиксации смещений требуется действующий `member_id` и актуальный `generation_id`; при несоответствии возвращается ошибка.
5. **FetchCommittedOffsets**: возвращает последние зафиксированные смещения для заданных пар (`group_id`, `partition`).
6. Если потребитель не отправлял `heartbeat` дольше `session timeout` (по умолчанию 30 секунд), координатор исключает его из группы и инициирует ребалансировку.

4.1.6. Алгоритм диапазонного назначения партиций

Для каждой темы: партиции сортируются по `partition_id`, члены группы сортируются по `member_id`. Диапазон `partition_ids` делится на `len(members)` равных частей; остаток (если число партиций не кратно числу членов) прибавляется к первым членам. Результат — список (`member_id`, `[partition_ids]`).

4.1.7. Аутентификация

Клиент передаёт токен в заголовке gRPC-метаданных `bunnumq-auth-token`. Сервер проверяет токен по списку, заданному в конфигурации при запуске. Если список пуст — аутентификация отключена (режим `PLAINTEXT`). Недействительный токен возвращает `Unauthenticated`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.2. Требования к надёжности

1. При отказе одного узла из трёх кластер продолжает обслуживать запросы (кворум Raft).
2. Данные, подтверждённые с `acks=all`, не теряются при отказе одного узла.
3. Узел после перезапуска восстанавливает состояние Metadata FSM из snapshot и replaying команд Raft; Partition FSM восстанавливает Storage из журнала на диске.
4. Storage корректно обрабатывает ситуацию torn write (незавершённая запись последнего пакета): при открытии активного сегмента хвост усекается до последней полной записи.
5. Очистка сегментов по политике хранения (`DeleteSegmentsBefore`) реализована как команда Raft, что гарантирует одинаковое удаление данных на всех репликах.
6. Операция graceful shutdown: узел прекращает приём новых RPC, ожидает завершения текущих, синхронизирует Storage и корректно завершает процесс dragonboat.

4.3. Требования к составу и параметрам технических средств

Таблица 4.1 — Минимальные требования к аппаратным средствам

Параметр	Минимальное значение
CPU	2 ядра x86-64 на узел
Оперативная память	512 МБ на узел (без учёта данных)
Дисковое хранилище	SSD или HDD; объём определяется <code>retention_bytes</code> × число партиций
Операционная система	Linux (ядро 5.4+) или macOS 12+
Сеть	TCP-связность между всеми узлами; задержка < 100 мс

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.4. Требования к информационной и программной совместимости

1. Язык реализации: Go 1.25 и выше.
2. Внешний транспорт: gRPC (HTTP/2), Protobuf. Совместимость на уровне бинарного протокола с любым gRPC-клиентом, использующим сгенерированные из `api/*.proto` stub-файлы.
3. Протокол Raft: библиотека dragonboat v4 (github.com/ltni/dragonboat/v4); взаимодействие между узлами только через dragonboat, сторонние реализации Raft несовместимы.
4. Метрики: формат Prometheus text/plain, совместимый с Prometheus Server 2.x и OpenMetrics.
5. Формат журнала (log): хранение совместимо с форматом пакетов Apache Kafka; это позволяет в будущем переключить транспорт без изменения слоя хранения.
6. Зависимостей на внешние базы данных, очереди или объектные хранилища нет; всё хранится локально на диске каждого узла.

4.5. Специальные требования

1. Partition FSM (IOndiskStateMachine) строго детерминирована: метод `Update()` не вызывает `time.Now()`, системных вызовов без гарантированного результата и сетевых операций.
2. Логирование: структурированный JSON через go.uber.org/zap; уровни `debug`, `info`, `warn`, `error`.
3. Отсутствие глобального состояния между запросами; конкурентность управляется через `goroutine` и `channel`.
4. Метрики экспортируются на HTTP-эндпоинте `/metrics`; порт настраивается в конфигурации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Состав программной документации

Разработка BunnyMQ сопровождается следующей программной документацией в соответствии с ГОСТ 19.201–78:

1. **Техническое задание** (ГОСТ 19.201–78) — настоящий документ; определяет требования к функциональным характеристикам, условиям эксплуатации, составу технических средств и порядку приёмки.
2. **Текст программы** (ГОСТ 19.401–78) — ссылка на репозиторий с исходным кодом брокера, клиентской библиотеки и утилиты командной строки.
3. **Пояснительная записка** (ГОСТ 19.404–79) — обоснование архитектурных и технических решений; описание алгоритмов хранения журнала, протокола Raft, работы FSM; выбор технологий и программных средств; ожидаемые технико-экономические показатели.
4. **Руководство оператора** (ГОСТ 19.505–79) — инструкции по сборке, настройке конфигурационного файла, запуску кластера, работе с `bunnymq-cli`; описание параметров окружения и диагностических сообщений.
5. **Программа и методика испытаний** (ГОСТ 19.301–79) — описание объекта и цели испытаний, применяемых средств тестирования, порядка проведения и методов проверки каждого требования ТЗ.

5.2. Специальные требования к документации

5.2.1. Требования к оформлению

- Все документы выполняются в соответствии с ГОСТ 19.106–78.
- Документы оформляются на листах А4 с полями: левое не менее 30 мм, правое не менее 10 мм, верхнее и нижнее не менее 20 мм.
- Шрифт Times New Roman 12 pt или аналог, межстрочный интервал 1,5.
- Нумерация разделов, подразделов, таблиц и рисунков согласно ГОСТ 19.106–78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- Пояснительная записка загружается в систему проверки Антиплагиат ВШЭ не позднее чем за один рабочий день до защиты.

5.2.2. Требования к содержанию пояснительной записки

Пояснительная записка должна включать:

- описание предметной области и постановку задачи;
- обоснование выбора алгоритма консенсуса Raft и библиотеки dragonboat;
- описание структуры сегментированного журнала: Storage, SegmentStorage, LogSegment, OffsetIndexSegment, TimeIndexSegment;
- описание работы Partition FSM и Metadata FSM;
- обоснование выбора gRPC и Protobuf в качестве транспорта;
- описание алгоритма диапазонного назначения партиций в группах потребителей;
- описание механизма long-poll fetch и уведомлений через newDataCh;
- результаты бенчмарков записи на диск.

5.2.3. Требования к руководству оператора

Руководство оператора должно включать:

- описание конфигурационного файла и всех параметров;
- порядок запуска кластера из N узлов ($N \geq 3$);
- описание команд `bunnymq-cli`;
- примеры типовых операций через клиентскую библиотеку;
- описание метрик Prometheus и диагностических сообщений;
- порядок graceful shutdown узла.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Экономическая эффективность

Разработка выполняется в рамках учебного курсового проекта; коммерческая оценка стоимости не является целью ТЗ. Тем не менее использование BunnyMQ как решения с открытым исходным кодом имеет измеримые преимущества по сравнению с коммерческими альтернативами.

6.1.1. Снижение лицензионных затрат

Существующие промышленные брокеры сообщений с сопоставимыми гарантиями доставки (Apache Kafka в managed-варианте, Amazon SQS / Amazon MSK, Confluent Cloud) имеют стоимость эксплуатации от нескольких сотен до десятков тысяч долларов в месяц в зависимости от объёма трафика. BunnyMQ распространяется с открытым исходным кодом, что позволяет развернуть его на собственной инфраструктуре без лицензионных отчислений.

6.1.2. Снижение требований к инфраструктуре

Зависимости BunnyMQ ограничиваются одним исполняемым файлом на узел и дисковым хранилищем. Отсутствует необходимость в отдельном ZooKeeper-кластере (в отличие от Apache Kafka версий до 3.x), внешней базе данных или объектном хранилище. Это снижает операционную сложность и стоимость инфраструктуры.

6.1.3. Прогнозируемые технические характеристики

Таблица 6.1 — Ожидаемые технические характеристики BunnyMQ

Характеристика	Ожидаемое значение
Пропускная способность (acks=0, последовательная запись)	$\geq 100\,000$ сообщ./с на узел (размер 1 КБ)
Пропускная способность (acks=all, RF=3)	$\geq 10\,000$ сообщ./с на кластер
Задержка записи (p99, acks=all, RF=3, LAN)	≤ 50 мс
Задержка чтения (cached, long-poll timeout 0)	≤ 5 мс
Коэффициент готовности кластера (3 узла)	$\geq 0,999$ (переживает отказ одного узла)
Максимальный объём хранимых данных на узел	ограничен только ёмкостью диска

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Показатели производительности определены расчётным методом на основе бенчмарков последовательной записи на SSD (результаты в `benchmarks/disk_test.go`) и характеристик библиотеки dragonboat v4.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Стадии и этапы разработки определены в соответствии с ГОСТ 19.102–77.

Таблица 7.1 — Стадии и этапы разработки BunnyMQ

Стадия разработки	Этап работ	Содержание работ	Сроки
Техническое проектирование	Анализ требований	Составление ТЗ, изучение Apache Kafka, dragonboat v4, согласование с руководителем	До 15.01.26
	Проектирование архитектуры	Разработка дизайн-документов: Storage, Raft FSM, координаторы, API, клиентская библиотека, группы потребителей	До 15.01.26
Рабочее проектирование	Разработка Storage (M1)	LogSegment, OffsetIndexSegment, TimeIndexSegment, SegmentStorage, Storage, retention, recovery; unit-тесты	До 01.03.26
	Разработка Raft FSM (M2)	NodeHost wrapper, Metadata FSM, Partition FSM, snapshot; тесты детерминизма	До 15.03.26
	Разработка координаторов и API (M3)	ClusterCoordinator, DataCoordinator, gRPC-серверы, auth, клиентская библиотека Producer/Consumer/AdminClient, bunnymq-cli	До 01.04.26
	Разработка групп потребителей (M4)	GroupCoordinator, JoinGroup/Heartbeat/LeaveGroup, CommitOffset, Consumer group mode в pkg/client	До 10.04.26
Внедрение и испытание	Интеграция и тестирование	Сборка кластера из 3 узлов, сквозные тесты, нагрузочное тестирование	До 20.04.26
	Подготовка документации	Пояснительная записка, руководство оператора, программа и методика испытаний, текст программы	До 25.04.26
	Сдача проекта	Защита курсовой работы, демонстрация работающего кластера	

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7.1. Сроки разработки и исполнители

Разработка выполняется одним исполнителем: Б.А. Багавиев, группа БПИ233. Программный продукт (программа и документация) должен быть завершён не позднее **30.04.2026**.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

8.1. Критерии приёмки

BunnyMQ считается готовым к приёмке при выполнении всех следующих условий:

1. Критерии качества кода:

- `go build ./...` завершается без ошибок;
- `go test ./...` проходит без ошибок на всех пакетах;
- `golangci-lint run ./...` не выдаёт критических предупреждений;
- покрытие кода тестами для пакетов `internal/storage` и `internal/metadata` не менее 80 %.

2. Критерии функциональности:

- все gRPC-методы Data API и Management API, перечисленные в разделе 4 ТЗ, реализованы и принимают запросы;
- кластер из 3 узлов корректно создаёт тему, принимает сообщения с `acks=all` и отдаёт их через `Fetch`;
- при аварийной остановке одного узла из трёх кластер продолжает обслуживать запросы;
- Metadata FSM корректно восстанавливается после перезапуска узла из `snapshot`;
- Storage корректно выполняет `recovery` после незавершённой записи (`torn write`).

3. Критерии надёжности:

- сообщения, подтверждённые с `acks=all`, не теряются при отказе одного узла;
- очистка по политике хранения (`DeleteSegmentsBefore`) применяется детерминированно на всех репликах через команду `Raft`.

4. Критерии производительности:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- пропускная способность последовательной записи на диск при размере блока 4 096 байт и без `O_SYNC` ≥ 500 МБ/с на тестовом стенде (подтверждается `benchmarks/disk_test.go`).

5. Критерии групп потребителей:

- группа из двух потребителей корректно делит партии темы через `JoinGroup`;
- при выходе одного потребителя (`LeaveGroup` или `session timeout`) производится ребалансировка без потери данных;
- зафиксированные смещения сохраняются в `Metadata FSM` и доступны после перезапуска потребителя.

8.2. Процедура приёмки

1. **Функциональное тестирование** — проверка каждого gRPC-метода через `bunnumq-cli` или автоматические тесты по сценариям из Программы и методики испытаний.
2. **Кластерное тестирование** — запуск трёх процессов `bunnumq` на локальной машине (разные порты); проверка `produce/fetch`, `failover` при убийстве одного узла, восстановления после перезапуска.
3. **Модульное тестирование** — запуск `go test ./...`; проверка результатов покрытия.
4. **Тестирование Storage** — проверка корректности записи и чтения, сегментного перехода, индексного поиска, восстановления после `torn write`.
5. **Бенчмарки** — запуск `go test ./benchmarks/... -bench=. -benchmem`; проверка соответствия характеристикам из таблицы 6.1.
6. **Проверка документации** — наличие всех документов состава, перечисленных в разделе 5 ТЗ.
7. **Составление акта приёмки** — по результатам составляется акт, подписываемый исполнителем и руководителем.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8.3. Требования к результатам приёмки

- `go test ./...` — статус PASS для всех пакетов.
- Отсутствие data-race при запуске `go test -race ./...`
- Покрытие `internal/storage` не менее 80 % по отчёту `go tool cover`.
- Кластер из 3 узлов успешно завершает сквозной тест: `CreateTopic` → `Produce(acks=all) × 100` → `Fetch` всех сообщений → проверка смещений.
- Отсутствие потери подтверждённых сообщений после `kill` одного узла и переизбрания лидера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ГЛОССАРИЙ И СОКРАЩЕНИЯ

Термины и сокращения, используемые в настоящем документе:

API — Application Programming Interface — программный интерфейс взаимодействия между компонентами системы.

Raft — Алгоритм консенсуса для распределённых систем, обеспечивающий согласованность реплицированного журнала при отказе меньшинства узлов.

dragonboat — Go-библиотека реализации алгоритма Raft (github.com/lni/dragonboat/v4), используемая в BunnyMQ для репликации партиций и метаданных кластера.

gRPC — Google Remote Procedure Call — высокопроизводительный RPC-фреймворк на базе Protocol Buffers и HTTP/2, используемый как транспорт Data API и Management API.

Protobuf — Protocol Buffers — язык описания схем и механизм бинарной сериализации данных от Google.

Shard — Группа Raft-репликации (в терминологии dragonboat). В BunnyMQ один shard соответствует одной партиции темы; плюс один shard для метаданных кластера.

FSM — Finite State Machine — абстракция dragonboat для применения команд Raft к состоянию. В BunnyMQ: `IStateMachine (Metadata FSM)` и `IOndiskStateMachine (Partition FSM)`.

Тема (Topic) — Логический поток сообщений с уникальным именем; разделяется на партиции для параллельной обработки.

Партиция (Partition) — Независимый упорядоченный подпоток сообщений темы. Порядок гарантируется только внутри одной партиции.

Смещение (Offset) — Монотонно возрастающий 64-битный идентификатор позиции сообщения в партиции.

Производитель (Producer) — Клиент, отправляющий сообщения в темы.

Потребитель (Consumer) — Клиент, читающий сообщения из партиций.

Группа потребителей (Consumer Group) — Совокупность потребителей, совместно читающих тему; каждая партиция обрабатывается ровно одним членом группы.

acks=all — Режим подтверждения записи: производитель ожидает кворума реплик. Реализован через `dragonboat.SyncPropose`.

acks=0 — Публикация без ожидания подтверждения. Реализован через `dragonboat.Propose`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Long-poll fetch — Режим чтения: запрос удерживается до появления новых данных или истечения `max_wait_ms`.

Сегментированный журнал — Структура хранения: журнал разбит на файловые сегменты фиксированного максимального размера.

mmap — Memory-mapped file — отображение файла в адресное пространство процесса. Используется для индексных файлов (`.index`, `.timeindex`).

Ребалансировка — Перераспределение партиций между членами группы потребителей при изменении её состава.

NotLeader — Ответ gRPC: узел не является лидером нужной партиции; содержит адрес текущего лидера.

Prometheus — Система сбора метрик в формате time-series с pull-моделью.

zap — Go-библиотека структурированного логирования `go.uber.org/zap`.

ЕСПД — Единая система программной документации — комплекс стандартов ГОСТ 19.xxx.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

СРАВНЕНИЕ С APACHE KAFKA

BunnyMQ проектировался с опорой на архитектуру Apache Kafka с рядом упрощений.

Таблица 2.1 — Сравнение BunnyMQ и Apache Kafka

Характеристика	BunnyMQ	Apache Kafka
Консенсус / репликация	Raft (dragonboat v4), multi-raft	ISR + KRaft (с 3.x)
Формат данных на диске	Kafka-совместимый batch format	Kafka batch format
Транспорт клиента	gRPC + Protobuf	Kafka binary protocol (TCP)
acks=0	Поддерживается	Поддерживается
acks=1	Не реализован	Поддерживается
acks=all	Поддерживается	Поддерживается
Транзакции	Не поддерживаются	Поддерживаются (с 0.11)
Идемпотентный Producer	Не поддерживается	Поддерживается
Группы потребителей	Range assignment, v1	Range, RoundRobin, Sticky, Cooperative
Cooperative rebalance	Не поддерживается	Поддерживается (с 2.4)
Изменение состава кластера в runtime	Не поддерживается	Поддерживается
Хранение смещений групп	Metadata FSM (Raft)	Внутренний топик __consumer_offsets
Мониторинг	Prometheus text/plain	JMX + Prometheus exporter

Примечание. Формат batch_data на диске в BunnyMQ совпадает с форматом пакетов Kafka, что обеспечивает потенциальную совместимость на уровне хранилища. Транспортный протокол (gRPC вместо Kafka binary protocol) несовместим: существующие Kafka-клиенты не могут подключиться к BunnyMQ напрямую.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПЕРЕЧЕНЬ gRPC-МЕТОДОВ

Все методы принадлежат пакету `bunmq.v1`. Аутентификация — заголовок `bunmq-auth-token` в gRPC-метаданных.

ManagementService

Таблица 3.1 — Методы ManagementService

Метод	Назначение	Тип
CreateTopic	Создание темы	Unary
DeleteTopic	Асинхронное удаление темы	Unary
ListTopics	Список тем кластера	Unary
DescribeTopic	Метаданные темы (лидеры, реплики)	Unary
AlterTopicPartitions	Увеличение числа партиций	Unary
AlterTopicRetention	Изменение параметров хранения	Unary
DescribeCluster	Состав кластера (узлы, адреса, статус)	Unary
ListPartitions	Партиции темы с метаданными	Unary

DataService

Таблица 3.2 — Методы DataService

Метод	Назначение	Тип
Produce	Запись пакета сообщений в партицию	Unary
Fetch	Чтение записей с long-poll	Unary
GetOffsets	Earliest / latest / by-timestamp смещения	Unary
JoinGroup	Регистрация потребителя в группе	Unary
Heartbeat	Подтверждение активности члена группы	Unary
LeaveGroup	Выход из группы	Unary
CommitOffset	Фиксация обработанных смещений	Unary
FetchCommittedOffsets	Чтение зафиксированных смещений группы	Unary

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Коды ошибок

Таблица 3.3 — Коды ошибок BunnyMQ

Код	Значение	gRPC status
OK	Успех	OK
InvalidArgument	Некорректные входные данные	INVALID_ARGUMENT
TopicNotFound	Тема не существует	NOT_FOUND
TopicAlreadyExists	Тема уже есть	ALREADY_EXISTS
PartitionNotFound	Партиция не существует	NOT_FOUND
NotLeader	Узел не лидер; ответ содержит адрес лидера	FAILED_PRECONDITION
OffsetOutOfRange	Смещение вне допустимого диапазона	OUT_OF_RANGE
MessageTooLarge	Размер пакета превышает допустимый	RESOURCE_EXHAUSTED
Unauthenticated	Токен отсутствует или недействителен	UNAUTHENTICATED
Unavailable	Нет лидера shard Raft	UNAVAILABLE
Timeout	Превышено время ожидания	DEADLINE_EXCEEDED

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4**СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ**

1. ГОСТ 19.101–77 Виды программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102–77 Стадии разработки // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103–77 Обозначения программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104–78 Основные надписи // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105–78 Общие требования к программным документам // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106–78 Требования к программным документам, выполненным печатным способом // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201–78 Техническое задание. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301–79 Программа и методика испытаний // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401–78 Текст программы // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404–79 Пояснительная записка // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505–79 Руководство оператора // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603–78 Общие правила внесения изменений // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
13. The Go Programming Language Specification. Версия 1.25. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

14. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC „14, 2014. [Электронный ресурс]. — URL: <https://raft.github.io/raft.pdf> (дата обращения: 26.04.2026).
15. dragonboat: A Multi-Group Raft library in Go. Документация dragonboat v4. [Электронный ресурс]. — URL: <https://github.com/lni/dragonboat> (дата обращения: 26.04.2026).
16. Apache Kafka Documentation. Версия 3.7. [Электронный ресурс]. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 26.04.2026).
17. gRPC Documentation. [Электронный ресурс]. — URL: <https://grpc.io/docs/> (дата обращения: 26.04.2026).
18. Protocol Buffers Documentation. Версия proto3. [Электронный ресурс]. — URL: <https://protobuf.dev/> (дата обращения: 26.04.2026).
19. Kreps J., Narkhede N., Rao J. Kafka: a Distributed Messaging System for Log Processing. NetDB „11, 2011. [Электронный ресурс]. — URL: <https://notes.stephenholiday.com/Kafka.pdf> (дата обращения: 26.04.2026).
20. Uber go/zap. Blazing fast, structured, leveled logging in Go. [Электронный ресурс]. — URL: <https://github.com/uber-go/zap> (дата обращения: 26.04.2026).
21. Prometheus Documentation. [Электронный ресурс]. — URL: <https://prometheus.io/docs/> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]